OFFICEFLOW

Office Management System

PROJECT TASK DOCUMENTATION

Assigned To	Sudharsanan
Project Type	Full-Stack Web Application
Technology Stack	MERN — MongoDB, Express.js, React.js, Node.js
Deliverable	Secure, documented, Git-submitted application
Deadline	To be confirmed by the Project Manager

Purpose of this document: Define the scope, technical expectations, security standards, documentation, and acceptance criteria for Sudharsanan's OfficeFlow assignment.

Prepared for internal project assignment and technical review

1. Project Overview

OfficeFlow is a centralized office management web application designed to manage daily business activities, team schedules, client information, quotations, invoices, payments, and expenses through a single platform.

The application must provide a responsive user interface, secure authentication, role-based access control, and integrated business workflows.

2. Project Objectives

- Develop a complete full-stack web application using the MERN stack.
- Implement secure authentication and authorization.
- Provide separate access permissions for Admin and Team Members.
- Build functional dashboards and business management modules.
- Integrate frontend, backend, and MongoDB database operations.
- Follow secure coding practices and maintain clean, reusable code.
- Document the application, APIs, setup instructions, and project architecture.
- Push the complete source code to a Git repository for review.

3. Functional Requirements

Module 1: Dashboard

- Display total clients, quotations, and invoices.
- Show revenue, pending payments, and recorded expenses.
- Display upcoming calendar events and recent activities.
- Provide quick actions for common operations.
- Populate statistics using actual database records.

Module 2: Authentication and Security

- Secure login and logout.
- Password hashing using bcrypt.
- JWT-based authentication.
- Protected frontend routes and backend API endpoints.
- Forgot password and password reset functionality.
- Input validation and appropriate error handling.
- Secure token and session management.

Module 3: User Management

- Allow Admin to create team member accounts.
- Store name, email, phone number, designation, and account status.
- Edit, activate, and deactivate team member accounts.
- Prevent unauthorized account creation and role changes.
- Allow users to manage their own permitted profile information.

Module 4: Role and Permission Management

Admin: Access authorized system modules, manage team member accounts, access business records and reports, and manage company settings and permissions.

Team Member: Access permitted modules, create and manage authorized records, and view assigned tasks and calendar events. Team Members must not create Admin accounts or elevate their own permissions.

All permissions must be enforced on the backend, not only hidden in the frontend.

Module 5: Common Calendar

- Day, week, and month views.
- Create, edit, view, and delete events.

- Event title, description, date, start time, and end time.
- Meeting location and online meeting link.
- Assign events to team members.
- Event categories and color coding.
- Recurring events and reminders.
- Shared team calendar.

Module 6: Client Management

- Create, view, edit, and delete client records.
- Store company name, contact person, email, phone, and address.
- Support GST and billing details where applicable.
- View client-related quotations and invoices.
- Maintain client activity history.

Module 7: Quotation Management

- Create quotations for clients and generate unique quotation numbers.
- Add products or services, quantities, and unit prices.
- Calculate subtotals, discounts, GST, and final totals.
- Configure validity dates and terms and conditions.
- Save drafts and update quotation statuses.
- Generate downloadable quotation PDFs and send quotations by email.
- Track Draft, Sent, Accepted, Rejected, and Expired statuses.

Module 8: Invoice Management

- Create invoices manually or from accepted quotations.
- Generate unique invoice numbers.
- Calculate quantities, discounts, taxes, and final totals.
- Set invoice dates and payment due dates.
- Generate downloadable invoice PDFs and send invoices by email.
- Track Paid, Unpaid, Partially Paid, and Overdue statuses.
- Maintain invoice history.

Module 9: Payment Management

- Record customer payments, including partial and full payments.
- Record payment method, date, and reference number.
- Calculate outstanding invoice balances.
- Maintain payment history.
- Update invoice payment status automatically.

Module 10: Expense Management

- Record business expenses with category, description, date, and amount.
- Upload supporting receipts where required.
- Filter expenses by date and category.
- Generate expense summaries.

Module 11: Reports and Analytics

- Revenue and expense summaries.
- Quotation status and conversion reports.
- Invoice and payment reports.
- Outstanding payment summaries.

- Date-range filters.
- Export reports to PDF or Excel.

Module 12: Notifications and Activity Logs

- Upcoming event reminders and invoice due-date notifications.
- Record important user activities.
- Track record creation and updates.
- Include timestamps and responsible user details.

Module 13: System Settings

- Company name, logo, and contact information.
- Default quotation and invoice settings.
- GST configuration where applicable.
- Quotation terms and conditions.
- User profile and password settings.

4. Technology Requirements

Layer	Technology
Frontend	React.js
Styling	Tailwind CSS
Backend	Node.js
API Framework	Express.js
Database	MongoDB
Authentication	JWT and bcrypt
Calendar	FullCalendar
PDF Generation	PDFKit or equivalent
Charts	Recharts
API Documentation	Swagger / OpenAPI
Version Control	Git and GitHub or another Git repository provider

Use JavaScript throughout the MERN application unless an alternative is approved.

5. Mandatory Security Requirements

- Hash passwords securely; never store plain-text passwords.
- Keep secrets, database credentials, and JWT keys in environment variables.
- Never commit .env files or credentials to the repository.
- Validate and sanitize user inputs on the server.
- Implement authentication middleware and role-based authorization.
- Check record ownership or access permissions where applicable.
- Use secure HTTP headers, appropriate CORS configuration, and rate limiting for sensitive endpoints.
- Prevent unauthorized access to Admin APIs.
- Configure secure token expiration and logout behavior.
- Protect password reset flows against unauthorized use.
- Avoid exposing stack traces, passwords, tokens, or sensitive information in API responses and logs.

- Use HTTPS in production and secure cookie settings if cookie-based authentication is implemented.
- Restrict file uploads by file type and size.
- Handle MongoDB queries safely and prevent injection attacks.
- Add appropriate loading states, validation messages, and error handling.
- Test unauthorized requests, invalid tokens, privilege escalation attempts, and invalid input scenarios.

6. Development and Code Quality Standards

- Use a clear frontend and backend folder structure.
- Separate routes, controllers, models, middleware, and business logic.
- Create reusable React components.
- Use consistent naming conventions and formatting.
- Implement proper HTTP status codes and consistent API responses.
- Add database validation and appropriate indexes.
- Avoid duplicate code and hardcoded business data.
- Ensure all important actions are connected to the backend.
- Maintain readable, maintainable, and reviewable code.
- Do not use mock data as a substitute for required working functionality in the final submission.

7. Required Project Documentation

The final submission must include a README.md containing:

1. Project overview and objectives.
2. Features and module descriptions.
3. Technology stack.
4. System architecture and folder structure.
5. Prerequisites and installation instructions.
6. Environment variable configuration using a safe .env.example.
7. MongoDB setup instructions.
8. Frontend and backend startup commands.
9. Authentication and role permission explanation.
10. API endpoint documentation or Swagger setup.
11. Testing instructions.
12. Known limitations and future enhancements.
13. Deployment instructions, if deployment is included.

Additional documentation should include database schema descriptions, key workflows, and screenshots of completed modules.

8. Git Repository and Final Submission

Sudharsanan must create and maintain a Git repository for the project.

Repository requirements

- Use meaningful commit messages and commit changes regularly.
- Maintain a .gitignore file.
- Include frontend and backend source code.
- Include required configuration templates and documentation.
- Exclude credentials, .env files, node_modules, build artifacts, and sensitive data.
- Push the final, tested code to the designated repository.
- Ensure the repository can be cloned and configured using the README instructions.

Final submission checklist

- Complete frontend source code.
- Complete backend source code.
- MongoDB models and database integration.
- Authentication and role-based authorization.
- Functional business modules.
- API documentation.
- README and .env.example.
- Test results and security checks.
- Screenshots of the completed application.
- Successful Git push and repository URL.
- Final project explanation and demonstration.

9. Project Explanation and Demonstration

At the end of the assignment, Sudharsanan must explain the project clearly and demonstrate the working application.

- Project purpose, objectives, and architecture.
- Frontend, backend, and database interaction.
- Authentication and role-based access control.
- Admin account creation and team member management.
- Calendar event creation and assignment.
- Client, quotation, and invoice workflows.
- Payment recording and expense tracking.
- Dashboard calculations and reports.
- API structure and database schema.
- Security measures and testing.
- Git repository structure, commit history, and setup instructions.

The developer must be able to explain the implementation and key technical decisions rather than only demonstrate the UI.

10. Completion and Acceptance Criteria

- All agreed core modules work correctly.
- Frontend and backend are integrated with MongoDB.
- Admin and Team Member permissions are verified.
- Quotations can be converted into invoices.
- Payments correctly update outstanding balances and invoice statuses.
- Dashboard figures reflect actual application data.
- Security checks and functional tests are completed.
- The application can be set up using the documented instructions.
- The source code is pushed to the designated Git repository.
- Documentation is complete and the final demonstration is delivered.

FINAL DELIVERABLE: A secure, functional, full-stack OfficeFlow application with source code, documentation, testing evidence, and a Git repository URL.

11. Assignment Details

Field	Details
Assigned Developer	Sudharsanan

Field	Details
Reviewer	Project Manager / Technical Lead
Deadline	To be confirmed by the Project Manager
Project Name	OfficeFlow — Office Management System
Stack	MERN — MongoDB, Express.js, React.js, Node.js